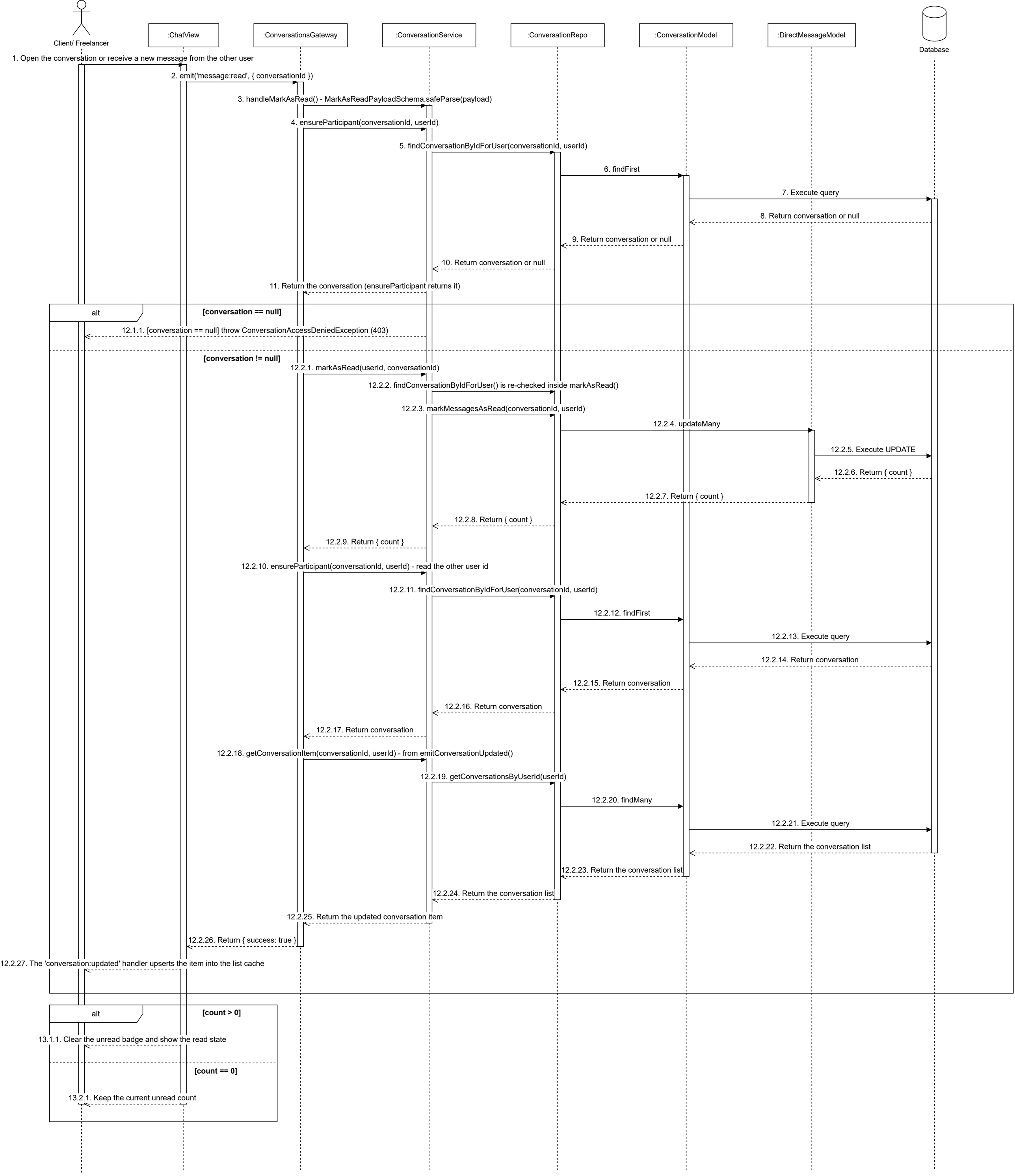

Client/ Freelancer
:ChatView
:ConversationsGateway
:ConversationService
:ConversationRepo
:ConversationModel
:DirectMessageModel
Database
1. Open the conversation or receive a new message from the other user
2. emit('message:read', { conversationId })
3. handleMarkAsRead() - MarkAsReadPayloadSchema.safeParse(payload)
4. ensureParticipant(conversationId, userId)
5. findConversationByIdForUser(conversationId, userId)
6. findFirst
7. Execute query
8. Return conversation or null
9. Return conversation or null
10. Return conversation or null
11. Return the conversation (ensureParticipant returns it)
alt
[conversation == null]
12.1.1. [conversation == null] throw ConversationAccessDeniedException (403)
[conversation != null]
12.2.1. markAsRead(userId, conversationId)
12.2.2. findConversationByIdForUser() is re-checked inside markAsRead()
12.2.3. markMessagesAsRead(conversationId, userId)
12.2.4. updateMany
12.2.5. Execute UPDATE
12.2.6. Return { count }
12.2.7. Return { count }
12.2.8. Return { count }
12.2.9. Return { count }
12.2.10. ensureParticipant(conversationId, userId) - read the other user id
12.2.11. findConversationByIdForUser(conversationId, userId)
12.2.12. findFirst
12.2.13. Execute query
12.2.14. Return conversation
12.2.15. Return conversation
12.2.16. Return conversation
12.2.17. Return conversation
12.2.18. getConversationItem(conversationId, userId) - from emitConversationUpdated()
12.2.19. getConversationsByUserId(userId)
12.2.20. findMany
12.2.21. Execute query
12.2.22. Return the conversation list
12.2.23. Return the conversation list
12.2.24. Return the conversation list
12.2.25. Return the updated conversation item
12.2.26. Return { success: true }
12.2.27. The 'conversation:updated' handler upserts the item into the list cache
alt
[count > 0]
13.1.1. Clear the unread badge and show the read state
[count == 0]
13.2.1. Keep the current unread count